Experiment-1(functions and arrays)

Q1) Write a program to create an UDF for input 10 numbers into a 1D array. Create two functions MAX() and MIN(). MAX() is used to return the largest element and MIN() is used to return the smallest number in array.

```
#include <stdio.h>
#define SIZE 10
int MAX(int a[], int n) {
   int max = a[0];
   for (int i = 1; i < n; i++)
      if (a[i] > max)
         max = a[i];
   return max;
}
int MIN(int a[], int n) {
   int min = a[0];
   for (int i = 1; i < n; i++)
      if (a[i] < min)
         min = a[i];
   return min;
}
int main() {
   int a[SIZE];
   for (int i = 0; i < SIZE; i++)
      scanf("%d", &a[i]);
   printf("Max: %d\n", MAX(a, SIZE));
   printf("Min: %d\n", MIN(a, SIZE));
   return 0;
}
```

Q2) Write a C program to create methods for operations insertion and display on 1D array of elements using UDF.

```
#include <stdio.h>

void display(int a[], int n) {

   for (int i = 0; i < n; i++)

      printf("%d ", a[i]);

   printf("\n");

}

void insert(int a[], int n, int ele, int pos) {

   for (int i = n; i > pos; i--)

      a[i] = a[i - 1];

   a[pos] = ele;

   display(a, n + 1);

}

int main() {

   int a[50], n, ele, pos;

   scanf("%d", &n);

   for (int i = 0; i < n; i++)

      scanf("%d", &a[i]);

   scanf("%d", &ele);

   scanf("%d", &pos);

   insert(a, n, ele, pos);

   return 0;

}
```

Q3) Write a C program to create methods for operations deletion, and display on 1D array of elements using UDF.

```
#include <stdio.h>

void display(int a[], int n) {

   for (int i = 0; i < n; i++)

      printf("%d ", a[i]);

   printf("\n");

}

int del(int a[], int n, int pos) {

   for (int i = pos; i < n - 1; i++)

      a[i] = a[i + 1];

   display(a, n - 1);

   return n - 1;

}

int main() {

   int a[50], n, pos;

   scanf("%d", &n);

   for (int i = 0; i < n; i++)

      scanf("%d", &a[i]);

   scanf("%d", &pos);

   del(a, n, pos);

   return 0;

}
```

Experiment 2: (concepts of matrix and sparse matrix)

Q1) Write a C program to create function for performing matrix multiplication using UDF

```
#include <stdio.h>
void mul(int a[10][10], int b[10][10], int r[10][10], int r1, int c1, int c2) {
   for (int i = 0; i < r1; i++)
      for (int j = 0; j < c2; j++) {
         r[i][j] = 0;
         for (int k = 0; k < c1; k++)
            r[i][j] += a[i][k] * b[k][j];
      }
}
void disp(int a[10][10], int r, int c) {
   for (int i = 0; i < r; i++) {
      for (int j = 0; j < c; j++)
         printf("%d ", a[i][j]);
      printf("\n");
   }
}
int main() {
   int a[10][10], b[10][10], r[10][10], r1, c1, r2, c2;
   scanf("%d %d", &r1, &c1);
   scanf("%d %d", &r2, &c2);
   if (c1 != r2)
      return 0;
```

```
    for (int i = 0; i < r1; i++)
        for (int j = 0; j < c1; j++)
            scanf("%d", &a[i][j]);
    for (int i = 0; i < r2; i++)
        for (int j = 0; j < c2; j++)
            scanf("%d", &b[i][j]);
    mul(a, b, r, r1, c1, c2);
    disp(r, r1, c2);
    return 0;
}
```

Q2) Write a C program to input elements into a square matrix and display the transpose of it using UDF.

```
#include <stdio.h>
void transpose(int a[10][10], int t[10][10], int n) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            t[j][i] = a[i][j];
}
void disp(int a[10][10], int n) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++)
            printf("%d ", a[i][j]);
        printf("\n");
    }
```

```
}
int main() {
   int a[10][10], t[10][10], n;
   scanf("%d", &n);
   for (int i = 0; i < n; i++)
      for (int j = 0; j < n; j++)
         scanf("%d", &a[i][j]);
   transpose(a, t, n);
   disp(t, n);
   return 0;
}
```

Q3) Write a program to input elements into a 4X4 matrix, check it for sparse or not. If sparse then store the non-zero elements into an alternate matrix and then display it using UDF.

```
#include <stdio.h>
int isSparse(int a[4][4]) {
   int z = 0;
   for (int i = 0; i < 4; i++)
      for (int j = 0; j < 4; j++)
         if (a[i][j] == 0)
            z++;
   return z > 8;
}
```

```
void store(int a[4][4], int b[][2], int *c) {
    *c = 0;
    for (int i = 0; i < 4; i++)
        for (int j = 0; j < 4; j++)
            if (a[i][j] != 0) {
                b[*c][0] = i;
                b[*c][1] = j;
                (*c)++;
            }
}
void disp(int b[][2], int c) {
    for (int i = 0; i < c; i++)
        printf("%d %d\n", b[i][0], b[i][1]);
}
int main() {
    int a[4][4], b[16][2], c;
    for (int i = 0; i < 4; i++)
        for (int j = 0; j < 4; j++)
            scanf("%d", &a[i][j]);
    if (isSparse(a)) {
        store(a, b, &c);
        disp(b, c);
    }
    return 0;
}
```

Experiment 3: (Pointer, structure and DMA)

Q1) Write a program to store N numbers using dynamic memory allocation and then find the largest element using UDF.

```
#include <stdio.h>

#include <stdlib.h>

int largest(int a[], int n) {

   int max = a[0];

   for (int i = 1; i < n; i++)

      if (a[i] > max)

         max = a[i];

   return max;

}

int main() {

   int *a, n;

   scanf("%d", &n);

   a = malloc(n * sizeof(int));

   for (int i = 0; i < n; i++)

      scanf("%d", &a[i]);

   printf("%d", largest(a, n));

   free(a);

   return 0;

}
```

Q2) Write a C program to create a structure called student to store your rollno, name, age. Create an array to input 5 students data and then create an UDF to display details where age>=20.

```
#include <stdio.h>

#include <string.h>

struct Student {

   int roll;

   char name[50];

   int age;

};

void display(struct Student s[], int n) {

   for (int i = 0; i < n; i++)

      if (s[i].age >= 20)

         printf("%d %s %d\n", s[i].roll, s[i].name, s[i].age);

}

int main() {

   struct Student s[5];

   for (int i = 0; i < 5; i++) {

      scanf("%d", &s[i].roll);

      getchar();

      fgets(s[i].name, 50, stdin);

      s[i].name[strcspn(s[i].name, "\n")] = 0;

      scanf("%d", &s[i].age);

   }

   display(s, 5);    return 0;     }
```

Q3) Write a program to create a structure for products of a super market. Store product no, name and cost for N products using dynamic memory allocation. Display the products whose cost is in between 100 rupees to 1000 rupees.

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

struct Product {

   int pno;

   char name[50];

   float cost;

};

void display(struct Product p[], int n) {

   for (int i = 0; i < n; i++)

      if (p[i].cost >= 100 && p[i].cost <= 1000)

         printf("%d %s %.2f\n", p[i].pno, p[i].name, p[i].cost);

}

int main() {

   struct Product *p;

   int n;

   scanf("%d", &n);

   p = malloc(n * sizeof(struct Product));

   for (int i = 0; i < n; i++) {

      scanf("%d", &p[i].pno);

      getchar();

      fgets(p[i].name, 50, stdin);
```

```
        p[i].name[strcspn(p[i].name, "\n")] = 0;
        scanf("%f", &p[i].cost);
    }
    display(p, n);
    free(p);
    return 0;
}
```